Report for Assignment 4, by Group 9

1 Project name: textual

Textual uses python to build Terminal User Interfaces (TUIs) with buttons, sidebars etc. Similar to an app.

Repository URL: https://github.com/Textualize/textual

Issue URL: Add indicators for the TabbedContent widget · Issue #4270 · Textualize/textual

(Group URL: https://github.com/DillwynKnox/textual)

2 Results from Tasks

2.1 Part 0: Overview of issue

Title: Add indicators for the TabbedContent widget

URL: https://github.com/Textualize/textual/issues/4270

Description: This issue is about adding indicators for the TabbedContent widget. These indicators are to let users know there are more tabs available when not all tabs can fit within the display.

Scope: This issue is non-trivial in the sense that it affects several classes, implements UI navigation, tab management, and visual changes i.e. an indicator in the shape of an arrow. The code affected, other than tests, can be found in _tabs.py.

2.2 Part 1: Setup

2.2.1 Onboarding experience

For Assignment 4, we chose a new project instead of continuing with the project from Assignment 3. The left the old project (BorgBackup) due to lack of suitable issues and the complexity of running the program. We switched to working with textual, as we managed to identify a suitable issue. The onboarding of textual was quite well documented in the read me and contribution file. It used a legacy version poetry to install from source, which required some

adjustment of commands when using a modern version of poetry. Installation worked for all three OS systems used by project members: Linux based, MacOS and Windows.

2.3 Part 2: Issue resolution

2.3.1 Requirements of the functionality

Requirements related to the functionality are identified and described in a systematic way. Each requirement has a name (ID), title, and description. The description can be one paragraph per requirement.

Core requirements

- **[CR1] Right Indicator:**
  - When not all tabs fit in the available width, the UI must show a **right indicator** if there are hidden tabs to the right.
- **[CR2] Left Indicator:**
  - When tabs are hidden on the left (scrolled), the UI must show a **left indicator**.
- **[CR3] No Indicator**
  - If all tabs fit, **no indicators** are shown.

Extra requirements

[ER1] Indicators update dynamically
Indicators must update when:

- the terminal is resized,
- the active tab changes (because `Tabs` may auto-scroll),
- tabs are added/removed,
- tabs are hidden/shown,
- tab labels change (relabel).

[ER2] Must work for `TabbedContent`

- Since `TabbedContent` uses `ContentTabs(Tabs)`, the indicators must work in `TabbedContent` without additional changes.

[ER3] Accessibility / behavior preservation

- Keyboard navigation (`left`/`right`) must continue to work.
- Clicking tabs must continue to work.
- No change to which tab is active; indicators are purely informational in the MVP.

Non-Functional Requirements:

- [NF1] Testing:
  - There are Snapshot tests for the existence of the Arrows.
  - There are unit tests for the added methods
- [NF2] Linting:
  - The changed code is linted according to the project description

Extraordinary Requirement

- [OR1] Clickable Arrows
  - The arrows are clickable and allow scrolling through the tabs by clicking on them

2.3.2 The issue is resolved

The issue was resolved according to the requests.

Non-Functional Requirements:

- [NF1] Testing:
  - There are Snapshot tests for the existence of the Arrows.
  - There are unit tests for the added methods
- [NF2] Linting:
  - The changed code is linted according to the project description

Extraordinary Requirement (P+)

- [OR1] Clickable Arrows
  - Indicators (arrows) do not only appear when there are more tabs hidden to the left or the right, they are also clickable.

2.3.3 Tests added according to the functionality requirements

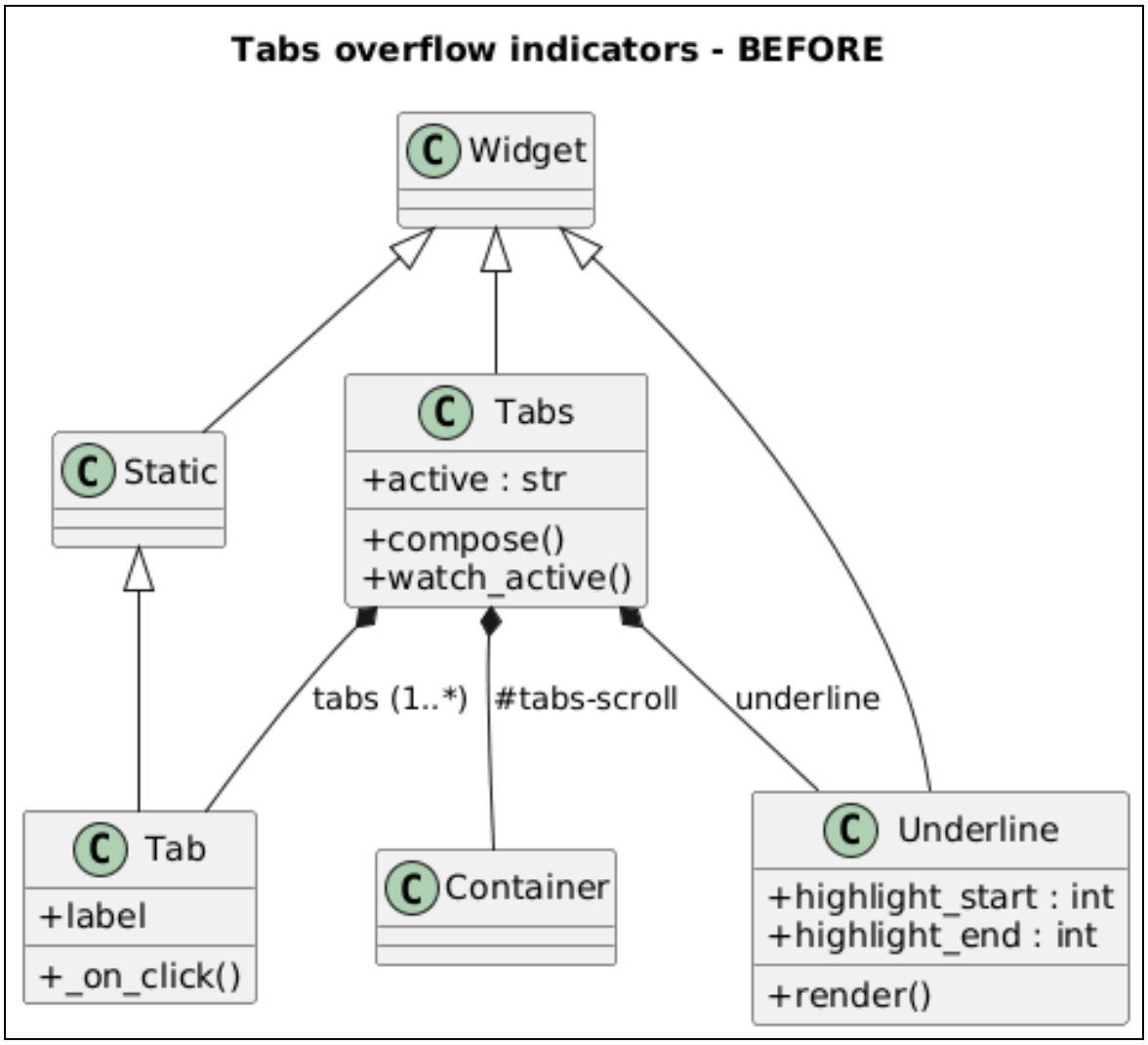
Tests added:

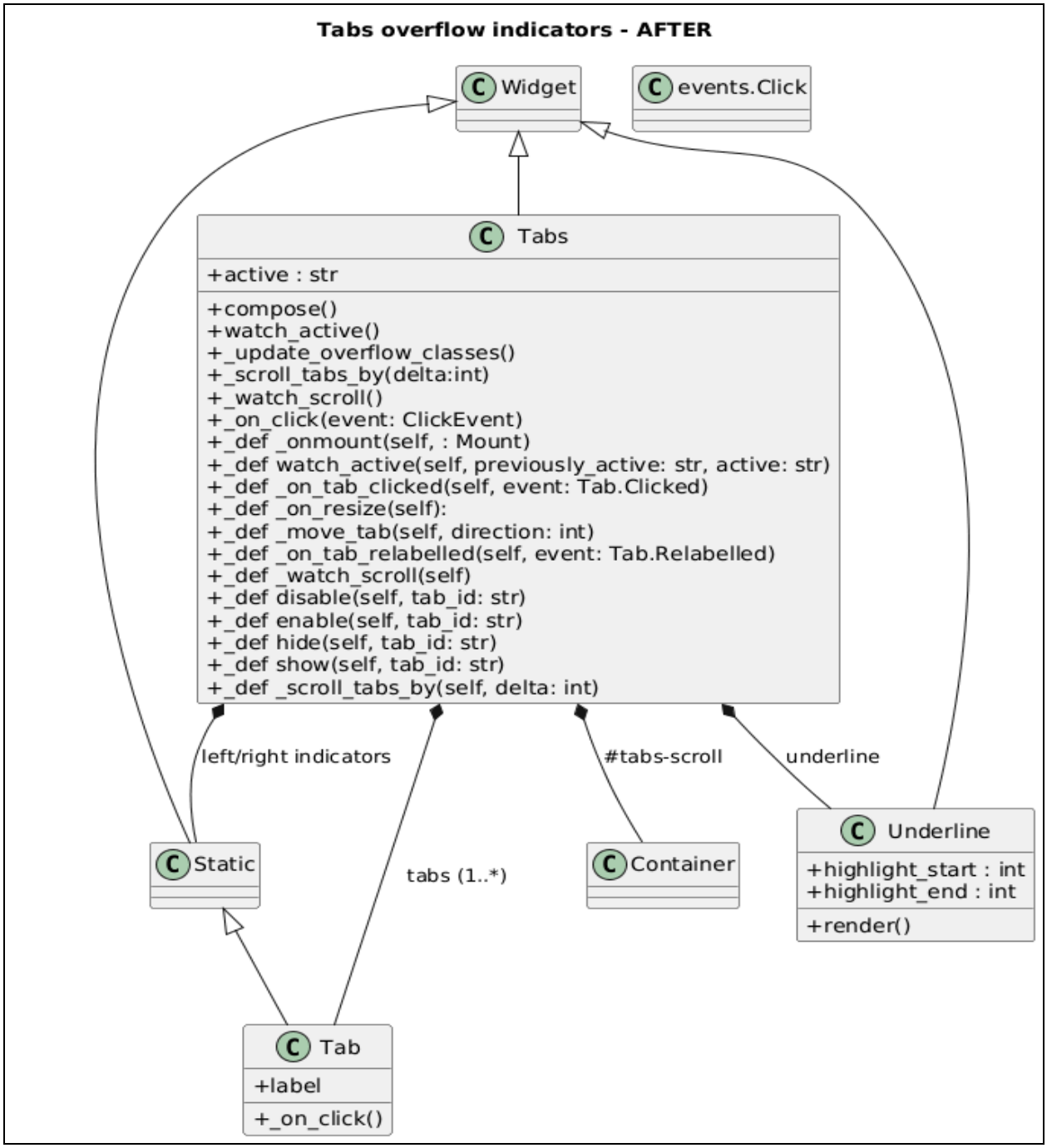
- In test_tabs.py, added tests for the new function `_overflow_classes` :
  - test_overflow_classes_has_no_overflow, test_overflow_classes_with_overflows_when_many_tabs, test_overflow_classes_after_scrolling.

    It is tested that when there are tabs hidden

- In test_tabs.py, added tests for the new function _wall_scroll:
    -

2.3.4 UML diagrams for the resolved issue





2.3.4 Patch

[(copy your changes or the add git command to show them)

https://github.com/DillwynKnox/textual/compare/original_main...main

Optional (point 4): the patch is clean, as evident form the patch.
Optional (point 5): considered for acceptance (passes all automated checks).]

2.3.5 Test Results before and after

[Overall results with link to a copy or excerpt of the logs (before/after refactoring).]

Test Logs before we changed anything: https://pastebin.com/gvJWKvXV
Test Logs with new Test but without functionality: https://pastebin.com/gkyxt9wd
Test Logs after we implemented the Issues: https://pastebin.com/CAL5TLaw

2.3.6 Relation to design patterns

Textual is intended to help developers build text-based apps that run in the terminal or in a browser. The concepts used are based on modern app development techniques (separating models, CSS styling etc) but it does have its own flavour. Textual generally works by creating widgets that have different looks and functionalities. It comes preloaded with several widgets. One of them, is a tabbed view. This means that you have a tab bar at the top and content below.

This is implemented by designing a widget for tab bars (called tabs) and a widget for a single tab (called tab). A separate widget, called ContentTabs, uses these widgets to create a functionality where tabs are linked to a content pane. The effect is that you click a tab and the correct content show up below.

We added a functionality so that the tab bar displays indicator arrows whenever the window is to small to show all tabs. There was a choice between implementing the changes in ContentTabs or in Tabs. The architecture as it was before implemented all functionaliry of the tabes in the_tabs.py file, a part from the linking of the tab to content. Thus, we implemented the arrows in the _tabs.py file.

Widgets in textual are built in a similar way as web apps, where functionality is separated from cosmetics by using textuals own flavour of CSS. We followed this architecture. In line with this, arrow indicators are always generated as widgets by the model. The model uses different tags to notify changes (such as “in focus”, or “enabled”). These tags are recognized by CSS to select what to display. In our case, we use tags to select if arrows should be displayed or not. In many ways, the class was already design to generate tabs that indicate if there are out-of-view tabs since it previously had a “center” function that ensured that the active tab was always centerd in the bar. We used some of these tags to both identify and implement the requirements for when arrow indicators should be shown and when not.

2.4 Part 3: Documentation

2.4.1 Essence standard v1.2 (page 52)

Table 8.8 – Checklist for Way-of-Working

State	Checklist
Principles Established	Principles and constraints are committed to by the team. Principles and constraints are agreed to by the stakeholders. The tool needs of the work and its stakeholders are agreed. A recommendation for the approach to be taken is available. The context within which the team will operate is understood. The constraints that apply to the selection, acquisition, and use of practices and tools are known.
Foundation Established	The key practices and tools that form the foundation of the way-of-working are selected. Enough practices for work to start are agreed to by the team. All non-negotiable practices and tools have been identified. The gaps that exist between the practices and tools that are needed and the practices and tools that are available have been analyzed and understood. The capability gaps that exist between what is needed to execute the desired way of working and the capability levels of the team have been analyzed and understood. The selected practices and tools have been integrated to form a usable way-of-working.
In Use	The practices and tools are being used to do real work. The use of the practices and tools selected are regularly inspected. The practices and tools are being adapted to the team's context. The use of the practices and tools is supported by the team. Procedures are in place to handle feedback on the team's way of working. The practices and tools support team communication and collaboration.
In Place	The practices and tools are being used by the whole team to perform their work. All team members have access to the practices and tools required to do their work. The whole team is involved in the inspection and adaptation of the way-of-working.
Working well	Team members are making progress as planned by using and adapting the way-of-working to suit their current context. The team naturally applies the practices without thinking about them. The tools naturally support the way that the team works. The team continually tunes their use of the practices and tools.

2.4.2 Effort spent in hours for contributions

Anton Trappe
Meetings: 10h (we had some trouble finding an appropriate project which took a lot of time and we made a detailed plan before starting)
Reading documentation: 30mins (documentation was clearly readable and it was easy to focus on the relevant part
Configuration and setup: 0 (on linux easy)
Analyzing code/output: 3h (understanding what happens where)
Writing documentation: 2h (mostly on the report)
Writing code: 4h (creating outline for basic arrows. Implementing CSS changes for arrows. Adding logic for calculating if the arrows are there, testing basic hide logic)
Running code: 0

Davide Mario Attebrant Sbrzesny
Meetings: 10h (Finding a project and designing and discussing a project plan took a lot of time)
Reading documentation: 4h (Understanding the design philosophy of textual apps as well as learning the syntax of its version of CSS)
Configuration and setup: 2h (Installed poetry, which in turn, required pipx. Trouble related to poetry no longer supporting the commands for creating a virtual environment led to time spent modifying the setup using the new commands for setting up and managing environments.)
Analyzing code/output: 6h (understanding the codebase enough to create a project plan and understand how to tackle the problem)
Writing documentation: 2h
Writing code: 2h (Most of the time was spent learning the new syntax and understanding the structure of the code, which I allocated to other points)
Running code: 0

Gabriel Alejandro Arias Goa
Meetings: 10h (Find project, decide an issue, project plan)
Reading documentation: 3h (Understand the logic of the code, where the files we needed are located, read the repository documentation)
Configuration and setup:1h(Follow the instructions in the repository, test the demo , install dependency)
Analyzing code/output: 3h (Analyse the test, see if some test not pass, see the differences parts of the program , see the classes)
Writing documentation: 2h
Writing code: 2h
Running code: 0

Mohamed Aziz Jribi
Meetings: 10 h
Reading documentation:2h
I read parts of the official Textual documentation to better understand the widget lifecycle, layout

system, and styling. The documentation was clear, so I could quickly focus on the relevant sections related to tabs and rendering.

Configuration and setup:0.5h

I worked on Ubuntu, and the setup was quite smooth. I installed Poetry and configured the environment without major problems.

Analyzing code/output 2h:

I spent time exploring the Textual codebase to understand how the TabbedContent widget is implemented and how tabs are updated during different events. This included identifying the methods and lifecycle events that affect tab rendering, scrolling, and layout.

Writing code and documentation:3h

My main contribution was to connect the overflow update logic to different lifecycle events. For every change in the tabs, the overflow indicators must be updated after the UI refresh. I modified several methods such as mounting, resizing, tab activation, clicking, moving, relabelling, scrolling, and visibility changes. In each relevant method, I added a call to update the overflow indicators so that the UI stays consistent.

Testing: 2h

I created unit tests to verify that the overflow indicators update correctly when tabs change. I also fixed and improved some existing tests related to issue #4270. In addition, I grouped the new tests in a dedicated folder and added comments to make their purpose clear.

Writing documentation: 1h

I contributed to writing the report, especially explaining the implementation choices and the testing strategy.

Yasmine Schüllerqvist

Meetings: 10h

Reading documentation: 1h

Configuration and setup: 1h (It took longer because I was confused initially, but a group member helped me in the end.)

Analyzing code/output: 3h (Trying to understand what is going on and what can be good things to test for new functionality.)

Writing documentation: 4h (I did a skeleton for the report, the reason it took so long is because I had trouble figuring out a good structure and redid it more than once.)

Writing code: 5h (It took a long time because I did some tests that were useless, but I redid them again so they worked.)

Running code: 0

In total, each group member spent at least 20 hours.

2.4.3 Overall Experience

If one thing became clear to us, it's that in software development, nothing is as simple as it seems. Implementing overflow indicators in the Tabs component seemed like a purely visual change, but it ended up being a lesson in connected systems. It wasn't just a matter of ‘drawing two arrows’; we had to tweak the Compose structure, wrestle with CSS, handle scroll reactively, and test everything. It helped us see that in modern frameworks, the user interface is entirely dependent on state management and event-driven design.

We learned the importance of studying the project before starting. Working with textual forced us to immerse ourselves in its architecture: understanding life cycles, reactive properties, and its styling system. We also discovered the difference between something looking good and working well. A component may look perfect on screen but break under certain logical conditions. Using a mix of snapshot tests and logic tests was key to understanding this distinction and leaving nothing to chance.